LLM-based Fuzzing Method Using UI-based Input Value Generation for IoT Devices

2025 IEEE 14th International Conference on Consumer Electronics - Berlin (ICCE-Berlin) | 979-8-3315-5381-4/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICCE-Berlin67488.2025.11277733

Hibiki Nakanishi*, Kento Hasegawa†, Seira Hidano†,
Kazuhide Fukushima†, Nozomu Togawa*
*Department of Computer Science and Communications Engineering, Waseda University
† KDDI Research, Inc.
Email: hibiki.nakanishi@togawa.cs.waseda.ac.jp, {kt-hasegawa, se-hidano, ka-fukushima}@kddi.com, ntogawa@waseda.jp

***Abstract*—The widespread adoption of IoT devices has raised concerns about security, especially as these devices increasingly handle sensitive data. Many modern IoT devices are operated via web-based user interfaces (UIs), which may block communications not via the UI by existing protocol fuzzing due to authentication mechanisms. To address this, we propose a UI-based fuzzing approach that generates test inputs using a large language model (LLM) and mutates the generated inputs. Evaluation experiments on an IoT device demonstrate the effectiveness of the proposed method in detecting vulnerabilities.**

***Index Terms*—IoT device, fuzzing, LLM, security, Web UI, Automated vulnerability testing**

I. Introduction

In recent years, IoT devices have been increasingly utilized in various fields such as smart homes, industrial automation, and healthcare, and their numbers are rapidly growing worldwide. As of 2024, the number of connected IoT devices globally has reached approximately 18 billion, and this figure is expected to increase to around 40 billion by 2033 [1]. IoT devices often handle highly confidential information over networks, making security a pressing concern [2]. Since the development of IoT devices heavily relies on third-party components for efficiency and cost-effectiveness [3], fuzzing which allows for black-box testing of products is considered effective.

Many modern IoT devices are designed to be operated through web-based user interfaces (UIs), and in some cases, they employ hash-based authentication for HTTP requests as a security measure. In such scenarios, communication attempts using existing protocol fuzzing may be rejected. To address this issue, we have proposed a method for automatically generating test inputs using a large language model (LLM) for the Web UIs of the target IoT devices referred to as the *LLM-based test input generation method* [4]. However, the method does not generate *mutated* test inputs and hence it does not transmit the test inputs effective for fuzzing to the target IoT device, either. In this paper, we propose a UI-based comprehensive fuzzing approach that incorporates mutation using an LLM. This method aims to effectively detect vulnerabilities even in cases where only communication through the Web UI is permitted.

This paper is organized as follows. In Section II, we introduce the IoT device fuzzing and the LLM-based test input generation method. In Section III, we describe the proposed method. In Section IV, we assess the effectiveness of the proposed method. Finally, Section V summarizes the paper.

II. Background

A. IoT device fuzzing

The recent proliferation of IoT devices has brought digital transformation to many fields, accelerating the exchange of not only personal information but also critical information. On the other hand, malicious attacks that exploit the vulnerabilities of IoT devices have been increasing, and there is a need to strengthen the security of IoT devices [5]. In particular, the increasing reliance on third-party components raises the possibility of vulnerabilities because their internal structure is unknown.

In addition, many IoT devices are operated by web-based operation interface (UI), which may contain vulnerabilities such as Cross Site Scripting (XSS) and SQL injection [6]. Vulnerabilities in UI are often exploited by external attacks, and the need for inspection is increasing.

B. LLM-based test input generation method

Figure 1 shows the flow of the LLM-based test input generation method, that automatically generates test input values for Web UI of IoT devices using LLM [4].

This method automatically generates test inputs which induce vulnerabilities using LLM for input forms obtained by analyzing the Web UI of IoT devices. It should be noted that this method also automatically completes error handling. For a specific example, if a string containing symbols is input to an input field that requires alphanumeric characters, an error message may be displayed and the submission of the value may be disabled. For effective inspections, error avoidance is quite important. When an error message is displayed, this method re-generate the test inputs that avoid the error with LLM. Vulnerability inspections are performed by repeating the generation and submission of test inputs as described above.

However, this method is generation-based fuzzing and does not include a mutation process [7]. Therefore, it may be inefficient when inspecting for some vulnerabilities, such as buffer overflow, which can be effectively detected by extending input values through mutation.

III. Proposed method

Based on the framework of [4], in this section, we propose a UI-based comprehensive fuzzing approach that incorpo-

Authorized licensed use limited to: Universitas Airlangga. Downloaded on October 07,2026 at 05:46:31 UTC from IEEE Xplore. Restrictions apply.

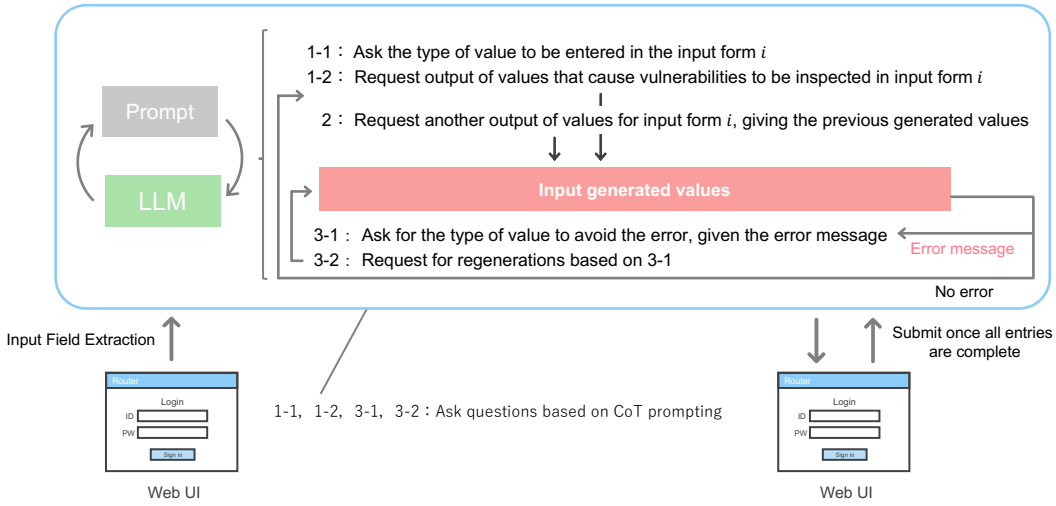


Fig. 1: Flow of LLM-based test input generation method [4].

rates mutation, using an LLM. Figure 2 shows the flow of the proposed method.

The proposed method is composed of the following three steps:

Step 1: HTML Analysis
Step 2: Mode Assignment
Step 3: Form Submission

After submitting the form in Step 3, the process returns to Step 1 and repeats the same procedure. The following sections explain each of these steps.

A. Step 1: HTML Analysis

In Step 1, the input fields are extracted by analyzing the HTML of the Web UI to extract input fields. The procedure of input fields extraction is described below.

1) Obtain HTML
2) Extract input tags
3) Extract the id attribute of each input tag

The extracted id attributes are used in the input value generation in Step 2.

B. Step 2: Mode Assignment

In Step 2, the process is stochastically assigned to the Generation mode (Mode G) and the Mutation mode (Mode M), and the process is executed in each mode. Mode G and Mode M each consist of two processes. If the UI has multiple input fields, the process for the assigned mode is repeated for each input field. First, the process for Mode G is described below.

G-1) The input values are automatically generated using LLM by specifying the name of the vulnerability to be inspected for the IoT device. The proposed method employs Chain-of-Thought (CoT) prompting, which is a prompting technique that encourages step-by-step thinking by providing multiple prompts to the LLM. Based on CoT, the first prompt asks the LLM to generate possible input values for an input field with a specific id attribute, and the second prompt asks the LLM to generate input values that are likely to cause the vulnerability to be inspected. The prompt template used is shown below.

G-1: Question 1

You will enter a value into the UI of a {test target}, one of the IoT devices. This input form has an id of {extracted id attribute}. Please output ONE example of values that can be entered here with ONE bullet beginning with *.

G-1: Answer 1

· · · (G-1: Answer for Question 1)

G-1: Question 2

So, what strings could cause {name of the vulnerability to be inspected} in such an input field? Please output data with bullet beginning with *. No explanation after *, just the content.

G-1: Answer 2

* Output 1
* Output 2
. . .

The generated values are input into the appropriate input fields.

G-2) If an input value is not formatted correctly, an error message may be displayed. When an error message is displayed, the LLM is asked to re-generate the input value to avoid the error with the content of the error message. The prompt template for regeneration is shown below.

G-2: Question 1

The following error message was returned: {error message}. What format should I use to avoid the error message?

Authorized licensed use limited to: Universitas Airlangga. Downloaded on October 07,2026 at 05:46:31 UTC from IEEE Xplore. Restrictions apply.

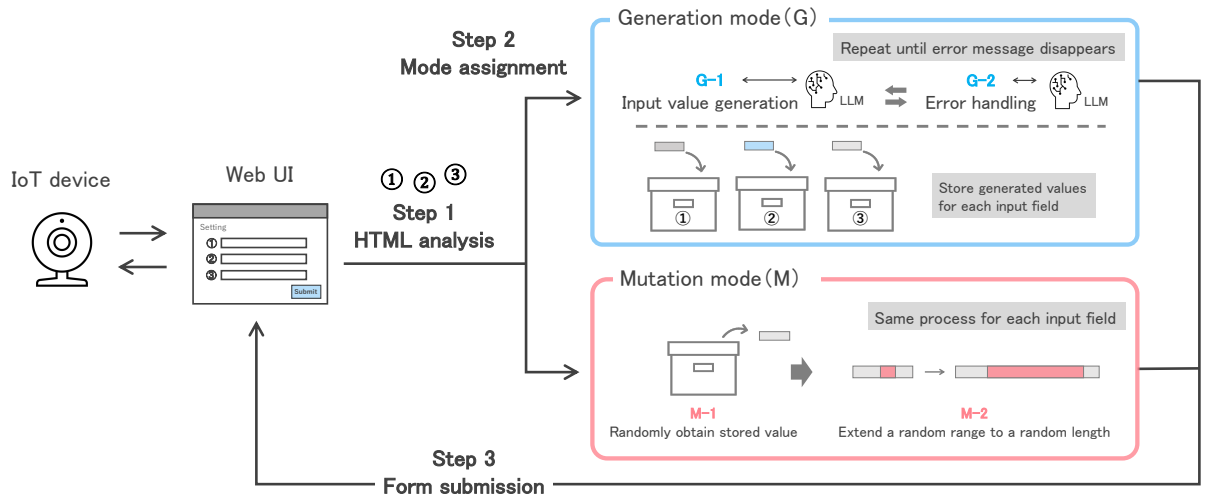


Fig. 2: Flow of the proposed method.

G-2: Answer 1

· · · (G-2: Answer for Question 1)

G-2: Question 2

Based on that, what string is likely to cause a {name of the vulnerability to be inspected} in this input field? If this field does not involve a {name of the vulnerability to be inspected}, generate another input value.

G-2: Answer 2

* Output 1
* Output 2
· · ·

The input values generated through G-1 and G-2 are saved for each input field of the UI.

The following describes the process for Mode M.

M-1) For the target input field, one of the input values generated and stored in Mode G is selected.

M-2) For a selected input value, an arbitrary range and length of portion is extracted and repeated for an arbitrary number of times to extend.

C. Step 3: Form Submission

Once values have been entered for all input fields, the submit button is automatically pressed.

IV. Experimental Evaluation

A. Experimental environments

We implemented the proposed method using Python 3.10.4. The proposed method was executed on a local PC running macOS Ventura 13.3.1 with an Apple M1 chip and 8 GB of RAM. As the LLM model, we used Mistral-7B-Instruct-v0.2 [8]. It was deployed in a Docker container on a GPU server, the specifications of which are shown in Table I. The model was accessed via FastChat v0.2.18 [9], and inference was performed using Transformers v4.41.0 [10].

TABLE I: Experimental environments.

Resource name	Model
OS	Rocky Linux 8.7
CPU	Intel Xeon Gold 5218*4
memory	251GB
GPU	NVIDIA A100 80GB*2

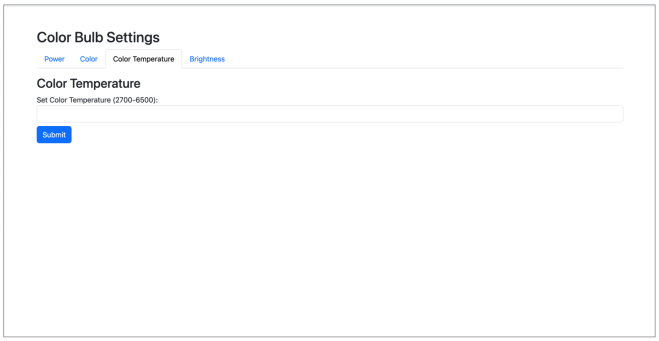


Fig. 3: Web UI to control the color bulb.

B. Fuzzing results

1) Effectiveness on back-end vulnerabilities: We implemented an IoT device based on a commercially available color bulb and a Web UI to control it on the local PC, as described in Section IV-A. Figure 3 shows the Web UI. On the IoT device, fuzzing using the proposed method was performed. To evaluate the effectiveness of the proposed method, buffer overflows were implemented in the IoT device. Considering the negative impact of the time required to generate LLM responses on the number of test values generated, we set the assignment ratio of Mode G and Mode M in Step 2 to 20% and 80%, respectively. In the Color Temperature setting of the device to be inspected shown in Figure 3, the following id attribute was extracted in Step 1.

- #colortemperatureValue

The actual prompt and LLM's responses for generating input values in Mode G are shown below.

Authorized licensed use limited to: Universitas Airlangga. Downloaded on October 07,2026 at 05:46:31 UTC from IEEE Xplore. Restrictions apply.

TABLE II: Crash detection time for the proposed method and LLM-based test input generation method.

	Time required for crash detection (sec)	
#	Proposed method	[4]
1	16.02	237.07
2	30.17	192.52
3	36.91	854.71

IV-B: Question 1

You will enter a value into the UI of a color bulb, one of the IoT devices. This input form has an id of #colortemperatureValue. Please output ONE example of values that can be entered here with ONE bullet beginning with *.

IV-B: Answer 1

2500: This value represents a warm white color temperature often used in indoor lighting.

IV-B: Question 2

So, what strings could cause buffer overflow in such an input field? Please output data with bullet beginning with *. No explanation after *, just the content.

IV-B: Answer 2

* “\x00”*n*
* “\x3c\x68\x6f\x6d\x65\x6e\x74\x6c\x6f\x00”
* “A”*1000
* “abcdefghijklmnopqrstuvwxyz”*100

Fuzzing was performed on the UI shown in Figure 3. In Mode M, the values generated in Mode G for the input fields were extended, and a crash due to buffer overflow was detected.

In order to verify the effectiveness of the proposed method, fuzzing using the LLM-based test input generation method [4] was also performed in the same environments. We measured the time required to detect a buffer overflow in the UI shown in Figure 3 three times for each method. Table II shows the time required for each method to detect the first buffer overflow crash. This measurement includes the time spent on LLM inference as part of the overall pipeline.

The experimental result shows that the buffer overflow can be detected by using the LLM-based test input generation method, but the time required to detect the buffer overflow can be significantly reduced by using the proposed method. Therefore, it can be confirmed that mutation is still highly necessary even when using an automated method based on LLM to automatically generate test values for effective inspection.

Notably, while we have verified the effectiveness of this method against buffer overflow, it is also effective against other vulnerabilities such as null pointer exception.

2) Effectiveness on front-end vulnerabilities: In section IV-B1, we verified the effectiveness of the proposed method against back-end vulnerabilities of the target IoT device. However, vulnerabilities may also exist in the front-end Web UIs that operate IoT devices. In this section, we verify the effectiveness of the proposed method on a front-end vulnerability using Damn Vulnerable Web Application (DVWA) [11], a vulnerable web application designed for educational purposes. The vulnerability to be tested is XSS. Figure 4 shows the Web UI of DVWA where XSS is intentionally implemented.

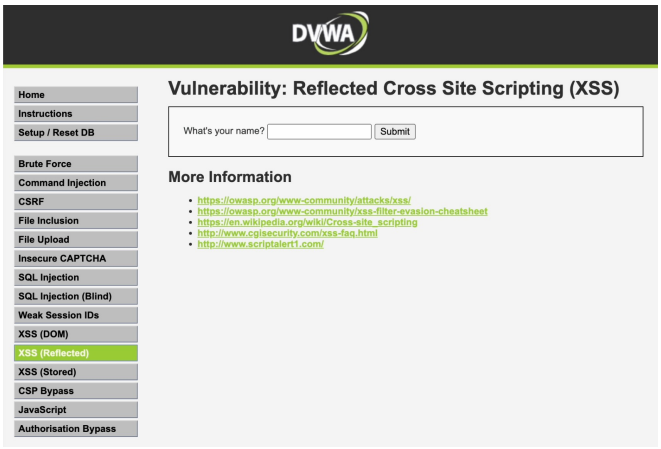


Fig. 4: Web UI of DVWA.

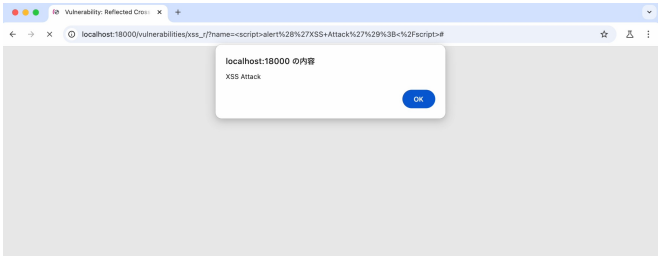


Fig. 5: Example of XSS detection.

The proposed method was applied to the Web UI shown in Figure 4, and the following input values were generated.

- <script>alert('XSS Attack');</script>
- <iframe src="malicious_website"></iframe>

The proposed method successfully detected XSS using those generated values. Figure 5 shows an example of the XSS detection.

C. Future work

In this paper, the proposed method only extends the arbitrary portion of the generated values for the mutation. To effectively detect vulnerabilities in a wide variety of IoT devices, it is desirable to increase the number of mutation types. In addition to extending, other possible mutations include replacing arbitrary portions with specific strings such as “on”, “off”, “null”, “True” and others, as well as bit-flipping. In addition to the random operations described above, LLM knowledge can also be utilized for mutation, which is expected to realize versatile and efficient inspection of a wide variety of IoT devices.

V. Conclusion

In this paper, we proposed a method to automatically generate test inputs based on the Web UIs of IoT devices using LLM and perform mutation-based fuzzing. The proposed method detected vulnerabilities in both the back-end and front-end of the IoT device, and the time required for

Authorized licensed use limited to: Universitas Airlangga. Downloaded on October 07,2026 at 05:46:31 UTC from IEEE Xplore. Restrictions apply.

detection is significantly shorter than LLM-based test input generation method [4]. For future work, it is desirable to increase the number of mutation types and to utilize the LLM knowledge in mutation, thereby improving the versatility of the method for a wider range of vulnerabilities and a wider variety of devices.

ACKNOWLEDGEMENT

The results of this research were obtained in part through a contract research project (JPJ012368C08101) sponsored by the National Institute of Information and Communications Technology (NICT).

REFERENCES

[1] statista, "Number of internet of things (iot) connections worldwide from 2022 to 2023, with forecasts from 2024 to 2033." https://www.statista.com/statistics/1183457/iot-connected-devices-worldwide/.

[2] Y. H. Hwang, "Iot security & privacy: threats and challenges," in *Proceedings of the 1st ACM workshop on IoT privacy, trust, and security*, pp. 1–1, 2015.

[3] B. Zhao, S. Ji, J. Xu, Y. Tian, Q. Wei, Q. Wang, C. Lyu, X. Zhang, C. Lin, J. Wu, *et al.*, "A large-scale empirical analysis of the vulnerabilities introduced by third-party components in iot firmware," in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, pp. 442–454, 2022.

[4] K. Hasegawa, H. Nakanishi, S. Hidano, K. Fukushima, K. Hashimoto, and N. Togawa, "Automated test input generation based on web user interfaces via large language models," in *Proceedings of the 10th International Conference on Internet of Things, Big Data and Security - IoTBDS*, pp. 297–304, INSTICC, SciTePress, 2025.

[5] I. Ahmad, M. S. Niazy, R. A. Ziar, and S. Khan, "Survey on iot: security threats and applications," *Journal of Robotics and Control (JRC)*, vol. 2, no. 1, pp. 42–46, 2021.

[6] S. Kals, E. Kirda, C. Kruegel, and N. Jovanovic, "Secubat: a web vulnerability scanner," in *Proceedings of the 15th international conference on World Wide Web*, pp. 247–256, 2006.

[7] X. Zhu, S. Wen, S. Camtepe, and Y. Xiang, "Fuzzing: a survey for roadmap," *ACM Computing Surveys (CSUR)*, vol. 54, no. 11s, pp. 1–36, 2022.

[8] "mistralai/mistralai/mistral-7b-instruct-v0.2." https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.2.

[9] PyPi, "Fastchat." https://pypi.org/project/fschat/.

[10] H. Face, "Transformers." https://huggingface.co/docs/transformers/index.

[11] A. K. Priyanka and S. S. Smruthi, "Webapplication vulnerabilities: Exploitation and prevention," in *2020 Second International Conference on Inventive Research in Computing Applications (ICIRCA)*, pp. 729–734, IEEE, 2020.

Authorized licensed use limited to: Universitas Airlangga. Downloaded on October 07,2026 at 05:46:31 UTC from IEEE Xplore. Restrictions apply.